III 1100

Chapter 1 - Binary Systems

Different numbering systems

- Decimal (Arabic): (0,1,2,3,4,5,6,7,8,9)
- Octal: (0,1,2,3,4,5,6,7)
- Hexadecimal: (0,1,2,3,4,6,7,8,9,A,B,C,D,E,F)
- Binary: (0,1)

1 - Positional Notation

$N = (a_{n-1} a_{n-2} \cdots a_1 a_0 . a_{-1} a_{-2} \cdots a_{-m})_r$

2 - Polynomial Notation

$N = a_{n-1} \times r^{n-1} + \cdots a_0 \times r^0 + a_{-1} \times r^{-1} + \cdots + a_{-m} \times r^{-m}$

$N = \sum_{i=-m}^{n-1} a_i r^i$

Conversions

Decimal → Anything

Integer part → keep dividing by the (of anything) base and collect the remainders in reverse order

fraction part → keep multiplying by base until it becomes "0". Collect integer parts (forward order).

Base 2^x → Binary

convert each digit into its x number of bits binary equivalent.

Binary → Base 2^x

split the number into groups of x and convert each group into their equivalent in Base 2^x

Anything → Decimal: Sum of polynomial notation

Binary addition

```
  1 1 1 1
+ 1010 0011 1011 0001
  0111 0100 0001 1001
1 0 001 0 111 0100 1010
```

Binary subtraction

```
  0   0   1   1
- 0 - 1 - 0 - 1
  0   1   1   0

  10 1011
- 01 0101
  01 0110
```

Binary codes: BCD

- used to represent decimal digits 0-9
- 4 bits are used

Ex:

$(9750)_{10} =$

$(1001\ 0111\ 0101\ 000)_{BCD}$

Binary codes: ASCII

Binary multiplication

```
  0   0   1   1
x 0 x 1 x 0 x 1
  0   0   0   1
```

Binary division

```
  0   1
÷ 1 ÷ 1
  0   1
```

Radix Complement (r's complement)

$[N]_r = r^n - (N)_r$

Diminished radix complement

$[N]_{r-1} = (r^n - 1)_r - (N)_r$

then replace 0's by 1's & 1's by 0's

Signed complement

a) Signed magnitude representation

1	0	0	0	0	0	1	1

-3 = 10000011 = 1/(Sign bit) 3000011

b) Signed 1's representation

1	1	1	1	1	1	0	0

-3 = 10000011 = 1/(Sign bit) 1111100

c) Signed 2's representation

1	1	1	1	1	1	0	1

-3 = 10000011 = 1/(Sign bit) 1111101

Substraction with 2's complement

$A - B = A + (-B)$

(add 2's complement of B to A)

Subtraction with 1's Complement

$A - B = A + [B]$

(add 1's complement of B to A)

Ex: A = 1010100 B = 1000011

A - B = A + [B] = (1010100) + (0111100) = (0010001)

1010100 + 0111100 = 0010000

+1 = 0010001

Overflow

* happens when the sum of the outside the range of the register

in signed complement overflow

- addition of two ⊕ #'s results in ⊖
- addition of two ⊖ #'s results in ⊕

Chapter 2 - Boolean algebra & Logic gate

Binary logic

1 - Variables: two discrete values

2 - Operations that assume logical meaning. Ex: is equivalent to Bool alg

Boolean algebra

1) Boolean variables: 1 & 0, True & False

2) Boolean operations:

Basic operations: AND, OR, NOT

Basic Boolean operations

AND: $x \cdot y$ or xy

x	y	xy
0	0	0
0	1	0
1	0	0
1	1	1

OR: $x + y$

x	y	x+y
0	0	0
0	1	1
1	0	1
1	1	1

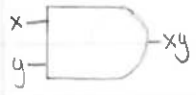
NOT

x	x'
0	1
1	0

Logic gates

Basic

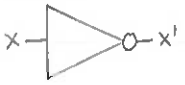
AND



OR



NOT



Boolean expressions (functions)

$f(x, y, z) = xy' + zx'$

name of function | Input variables

term: implementation with a gate

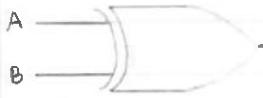
literal: any occurrence of an input variable or its complement.

NAND (universal)



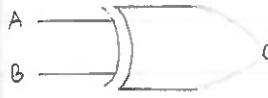
A	B	F
0	0	1
0	1	1
1	0	1
1	1	0

XOR



A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

XNOR



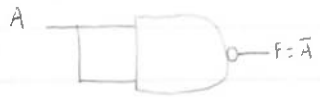
A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

NOR

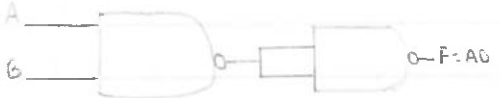


A	B	F
0	0	1
0	1	0
1	0	0
1	1	0

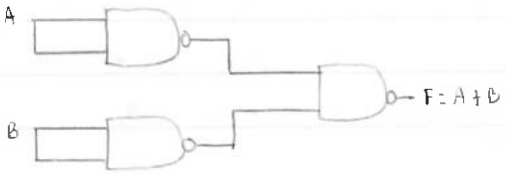
NAND universality



NOT gate



AND Gate



OR gate

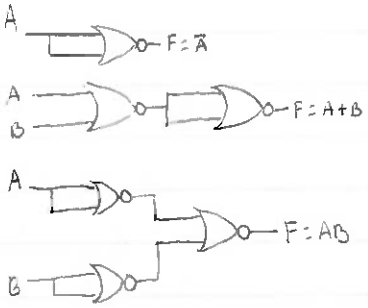
Truth table

2^n 2 to the power of n variables

of possible combinations of inputs.

Theorems of Boolean Algebra

1. $X+0=X$
2. $X \cdot 1 = X$
3. $X + X' = 1$
4. $X \cdot X' = 0$
5. $X+X=X$
6. $X \cdot X = X$
7. $X+1=1$
8. $X \cdot 0 = 0$
9. $(X')' = X$
10. $x+y = y+x$ commutative
11. $xy = yx$ commutative
12. $x+(y+z) = (x+y)+z$ associative
13. $x(yz) = (xy)z$ associative
14. $x(x+y) = xy + xz$ distributive
15. $x + yz = (x+y)(x+z)$ distributive
16. $(x+y)' = x'y'$ demorgan
17. $(xy)' = x' + y'$ demorgan
18. $x + xy = x$ absorption
19. $x(x+y) = x$ absorption



Standard sum of products (SOP)

Standard product of sums (POS)

minterm: combination in truth table

maxterm: combination in truth table

half-adder: two one-bit binary inputs, a carry out and a sum

full-adder: two one-bit binary inputs a carry in, a carry out and a sum.

half-adder



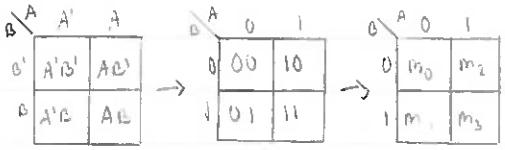
input		output	
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Chapter 3 - Level minimization

K-map (karnaugh map):

- four or less variables
- each square is a minterm
- 2^n squares

* don't cares can be 1's or 0's



Prime implicant: group of maximum adjacent squares.

Essential prime implicant: group that has at least one square that isn't covered by another group.

Optional prime implicant: group that has all its squares covered by another group.

NOR gate implementation

- function needs to be simplified into POS
- changing OR gates to NOR gates
- changing AND gates to NOR gates
- single literal going into the second-level gate must be complemented

multi-level implementation (NAND)

1. convert all AND gates to NAND
2. convert all OR gates to NAND
3. Check all the bubbles in the diagram.

Two-level implementation (NAND)

1. Simplify the function and express it in SOP
2. Draw a NAND gate for each product term that has at least 2 literals.
3. Draw a single gate using the AND invert or the invert OR in the second level, with inputs coming from outputs of first level gates.
4. A term with 1 literal requires an inverter in the first level. * If the single literal is complemented, it can be connected directly to an input of the second level NAND gate.

Chapter 4 - Combinational logic

Design circuit

1. Determine number of required inputs & outputs.
2. Derive truth table.
3. Obtain simplified Boolean functions.
4. Draw logic diagram & verify correctness.

Decoder

- Combinational circuit that converts binary info from n input lines to a maximum of 2^n unique output lines.
  → n-to-2^n decoder
- can be a minterm generator

2-4 decoder

2 inputs, 4 outputs

S1	S0	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

enable inputs

- in order to activate & deactivate the device
- EN=1 (is on so will behave normally)
- EN=0 (is off so nothing ever happens)

3-8 line decoder

- 3 inputs, 8 outputs

Inputs			Outputs							
x	y	z	D0	D1	D2	D3	D4	D5	D6	D7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

active-high decoders → generate minterms

active-low decoders → generate max terms

Encoder

- performs the inverse operation of a decoder
- has 2^n (or fewer) inputs and n outputs
- the output lines, as an aggregate, generate the binary code corresponding to each input value.

Ex. 8-to-3 encoder (8 inputs, 3 outputs).

- ambiguity: only one input can be active at any given time or else weird stuff happens.

Decoder expansions

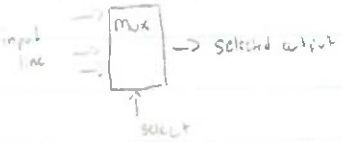
- larger decoders can be constructed using a number of smaller ones.
  → Hierarchical design
- Ex: a 4-to-16 decoder can be designed using five 2-to-4 decoders. (you can use one decoder as an enabler for the other 4 decoders.)

Priority encoder

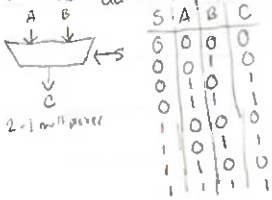
- if 2 or more inputs are equal to 1 at the same time, the input in the highest-numbered position will have precedence.
- valid out but indicated, is set to 1 only when one or more inputs are equal to 1.
  V= OR ogic of all inputs

Multiplexer (MUX)

- Selects one of several input signals and passes it on to the input.
- Routing of selected data input to output is controlled by select inputs



- Combination circuit with 2^n data inputs, 1 data output and a number of bit control input that select one of the data inputs



S	A	B	C
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$C = S'AB' + S'AB + SAB$
$= SA'B + SAB$
$= S'A(B' + B) + SB(A' + A)$
$= S'A + SB$

- rule to follow:
  2^n data inputs, 1 data output, n selectors

Demultiplexer (Demux)

- takes a single input & distributes it over several outputs.

ex: 1 line-to-8-line demultiplexer

1 input, 3 selectors, 8 outputs

* random disclamer!:

to add two n-bit numbers together, n full-adders should be cascaded.

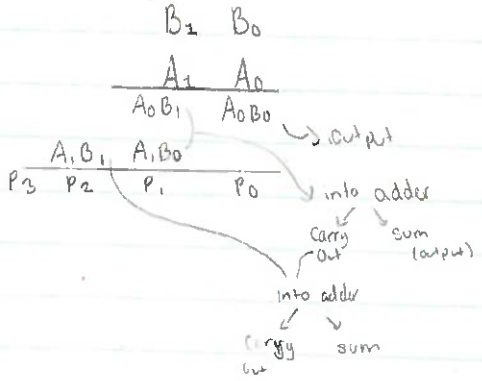
Label gate outputs

1 - label all gate outputs that are a function of input variables

2 - label gates that are a function of input variables & previously labeled gates

3 - repeat process until all outputs are labeled

Binary multiplier



Chapter 5 - Synchronous sequential logic

- digital system ← registers ← data processing units
- Binary info is transferred from one set of registers into another
- transfer can be direct or via a processing unit to perform an operation.

register

- group of Binary cells
- n cells can store n bits
- can be in 2^n states

Sequential logic circuits

1 - The input variables

2 - the previous states (internal states).

1 - time 2 - memory element

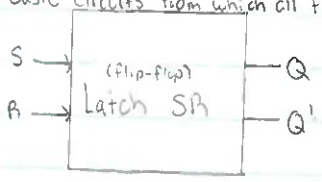
Flip-flop

- memory cell that stores one "bit" of info
- a set of these can be used to store several bits in a register.

2 different modes ← asynchronous: output reacts "immediately" to the changes of the input variables
← synchronous: changes of the input variables will become effective when the clock input is active.

Latches

- Basic circuits from which all flip-flops are constructed



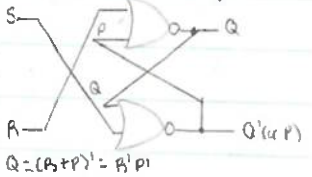
S is for set

R is for reset

Q' (complement state)

Q (normal state)

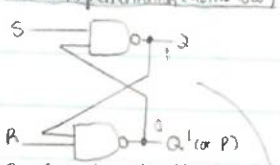
NOR implementation (active-high)



$Q = (R+P)' = R'P$

$P = (S+Q)' = S'Q'$

NAND implementation (active low)



$Q = (Q'S)' = (PS)'$

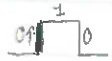
$Q' = (RQ)'$

clock signal

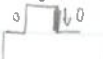
- positive level trigger



- positive transition trigger (positive edge)



- negative transition trigger (negative edge)



S	R	Q	Q'
1	0	1	0
0	1	0	1
0	0	1	0
0	0	0	1
1	1	?	?
1	1	0	0

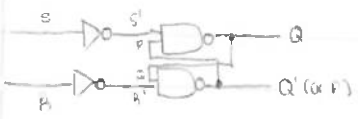
after SR: 10
after SR: 01
not allowed
10 01

S	R	Q	Q'
1	0	0	1
0	1	1	0
0	0		
0	0		
1	1	0	1
1	1	1	0

(not allowed)
after SR: 10
after SR: 01

NAND implementation

Active-high SR latch



S	R	Q	Q'	
1	0	1	0	
0	1	0	1	
0	0	1	0	after (SR=10)
0	0	0	1	after (SR=01)
1	1			(not allowed)
1	1	1	1	

$Q = (Q'S')' = (PS')' = P' + S = Q + S$

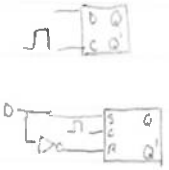
$Q' = (Q R)' = Q' + R$

Control input

- Controls when the state change can happen
- enable
- NAND latch

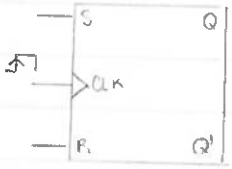
} SR Latch

D	C	Q	Q'
0	1	0	1
1	1	1	0
X	0	Q_n	Q_n'



SR Flip-flop characteristic table

C	S	R	Q_{n+1}	Q'_{n+1}	function
↑	0	0	Q_n	Q'_n	no change
↑	0	1	0	1	reset
↑	1	0	1	0	set
↑	1	1	0	0	forbidden
0	X	X	Q_n	Q'_n	inhibited



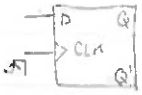
Q_n : state before positive edge

Q_{n+1} : state after positive edge

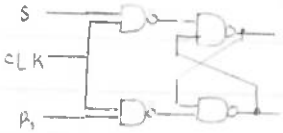
D Flip-flop

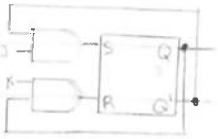
D	CLK	Q_{n+1}
0	(↑)	0
1	(↑)	1
X	0	Q_n

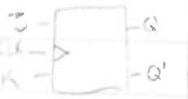
↑ characteristic table



JK Flip-flop







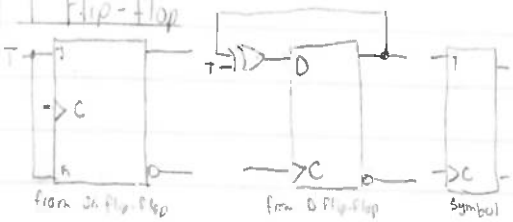
J	K	Q_{n+1}	Function
0	0	Q_n	no change
0	1	0	reset
1	0	1	set
1	1	Q'_n	complement (also toggle)

If $Q_n = 1$, $Q_{n+1} = 0$

If $Q_n = 0$, $Q_{n+1} = 1$

→ characteristic table

T Flip-flop

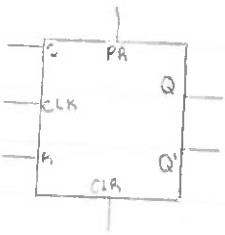


CLK	T	Q_{n+1}
0	X	Q_n (memorizing)
↓	0	Q_n (memorizing)
↓	1	Q_n' (toggle)

Asynchronous inputs

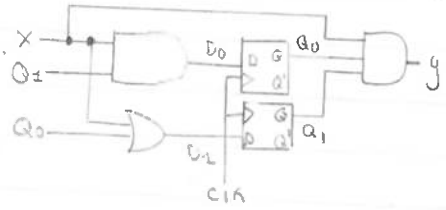
Def: inputs that operate independently of the synchronous inputs & clock.

- used to set the flip-flop to 1 or 0 state at any time.



Flip-flop state

- Behavior of synchronous sequential circuit can be determined from inputs, outputs & Flip flop state.



State equations:

Specifies the next state as a function of the present state & inputs.

State table:

- Sequence of outputs, inputs, & flip flop states
- present state indicates current value of flip flops
- next state indicates state after next clock edge
- output is output value on current clock edge

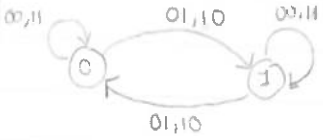
Present State	next state x=0	next state x=1	output x=0	output x=1
0 0	00	10	0	0
0 1	10	10	0	0
1 0	00	11	0	0
1 1	10	11	0	1

$Q_0(t+1) = D_0(t) = X(t)\,Q_1(t)$

$Q_1(t+1) = D_1(t) = X(t) + Q_0(t)$

State diagram

- Circles indicate next state
- Arrows point to next state



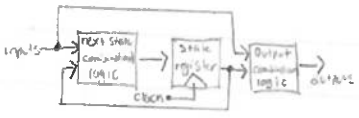
ex

present state	inputs		next state
A	x	y	A
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

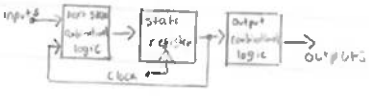
Mealy & Moore models

model of sequential circuits: inputs, outputs, internal states

mealy machine



moore machine



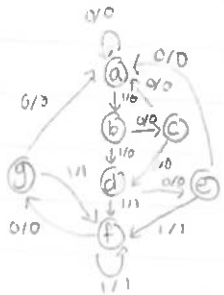
State reduction

- procedures for reducing the number of states in a state table, while keeping the external input-output requirements unchanged

present state	next state x=0	next state x=1	output x=0	output x=1
a	i	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
-> e	a	f	0	1
f	g	f	0	1
-> g	a	f	0	1

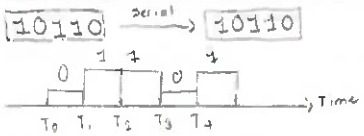
present state	next state x=0	next state x=1	output x=0	output x=1
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
-> d	c	f	0	1
e	a	f	0	1
-> f	e	f	0	1

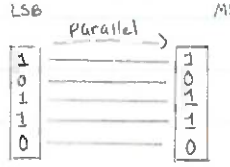
a	a	b	0 0
b	c	d	0 0
c	a	d	0 0
d	e	d	0 1
e	a	d	0 1

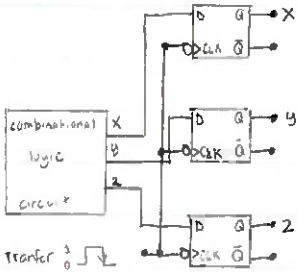


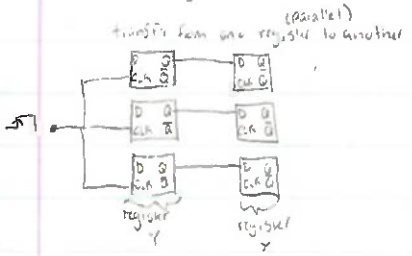
Chapter 6 - Registers & counters

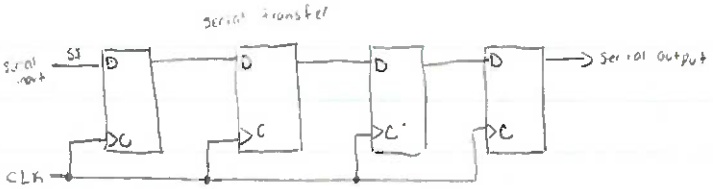
- Multiple flip-flops can be combined to form a data register
- Shift registers allow data to be transported one bit at a time
- Registers also allow for parallel transfer (many bits transfered at the same time)







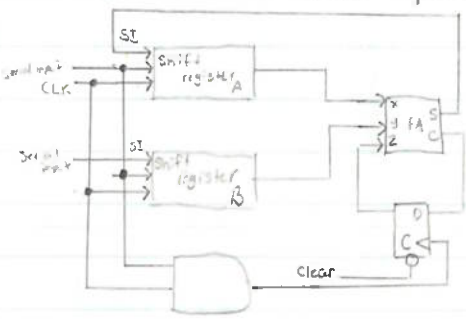




Serial transfer

Time	A	B
T0	1011	0011
T1	1101	1001
T2	1110	1100
T3	0111	0110
T4	1011	1011

Serial addition (D Flip Flop)



Sequential circuit: design procedure

1. From the word description and specifications of the desired operation, derive a state diagram for the circuit.
2. Obtain the state table
3. Reduce the number of states if necessary
4. Assign binary values to the states
5. Determine the number and types of flip-flops to be used
6. Obtain simplified flip-flop input equations and output equations
7. Draw the logic diagram

Excitation table JK Flip-Flop

J-K Flip-Flop			
Present	Next	Input J	K
0	→ 0	0	X
0	→ 1	1	X
1	→ 0	X	1
1	→ 1	X	0

Designing a JK Serial adder

- $J_Q = x \cdot y$
- $K_Q = x' \cdot y' = (x+y)'$
- $S = x \oplus y \oplus Q$



Present State (Q)	inputs x	y	next State (Q)	output (S)
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Counters

Def: register that goes through a prescribed series of states

types:
- asynchronous (ripple) → flip-flop output serves as a clock for triggering connected flip-flops
- synchronous → all flip-flops are triggered by a clock signal at the same time.

Counter design procedure

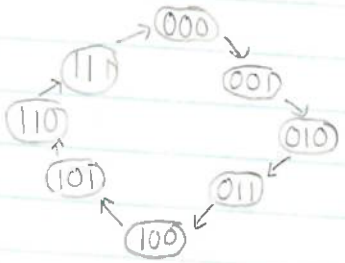
1 - Draw state table & diagram of output sequence
2 - Determine F-F type & make transition table
3 - Build state table with F-F inputs indicated
4 - Use K-maps to record & simplify the available outputs to create inputs.
5 - Implement the circuit.

Binary ripple counter

- Circuit that counts clock pulses & displays the running total in binary
- each F-F controls the CLK input of the next F-F
- F-F do not change states in exact synchronism with the applied clock pulse
- rippling effect (kinda like a domino)

A_3	A_2	A_1	A_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0

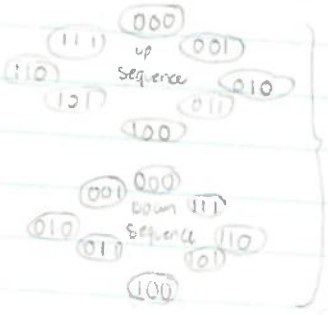
* reset signal sets all outputs to 0.



* number of output states is called the modulus or mod.

full sequence count: natural count that includes all possible binary numbers
modulus = max modulus

Truncated sequence count: modulus < max modulus less than all possible binary numbers are used



state diagram

Circuit			next		
Q_c	Q_b	Q_a	Q_c	Q_b	Q_a
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	0	0	0

state table

Transition tables

D - Flip-Flop

present	next	input
0	0	0
0	1	1
1	0	0
1	1	1

JK - Flip-Flop

Present	next	Input J	Input K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Building a K-map

1 - State diagram of output sequence

2 - State table of output sequence

3 - transition table for F-F we want to use

- Present & next outputs
- necessary inputs to create next outputs

Present $Q_C Q_B Q_A$	next $Q_C Q_B Q_A$	input $D_C D_B D_A$
0 0 0	0 1 0	0 1 0
0 1 0	1 1 0	1 1 0
1 1 0	0 0 1	0 0 1
0 0 1	0 0 0	0 0 0

Q_A

$Q_C Q_B$	0	1
0 0	0	0
0 1	1	x
1 1	0	x
1 0	x	x

Q_A

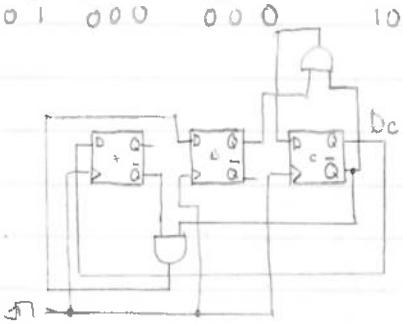
$Q_C Q_B$	0	1
0 0	1	0
0 1	1	x
1 1	0	x
1 0	x	x

Q_A

$Q_C Q_B$	0	1
0 0	0	0
0 1	0	x
1 1	1	x
1 0	1	x

D_C D_B D_A

$D_C = Q_B Q_C'$ $D_L = Q_A' Q_C'$ $D_A = Q_C$



Binary synchronous counters

Binary counter with JK flip-flop

- Build state diagram
- Build state table consisting of
  - current state output
  - next state output
  - JK inputs for each flip-flop
- Use K-maps to simplify expressions for JK inputs.
- Build the circuit for the counter

Ex:

0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7

3-bit binary counter

- 3 JK flip-flops needed
- Current state & next state outputs are 3 bits each
- 3 pairs of JK inputs

all states possible

Synchronous BCD

- 4 bits for each number

0000 -> 0001 -> 0010 -> 0011 -> ...

Same design process as others

Ex:

0 -> 3 -> 5 -> 7 -> 6 -> 0

- Same design process as before
- for missing states use don't cares.

don't cares for the next state & inputs

Ring counter

- takes serial output of the last Flip-Flop of a shift register & provides it to the serial input of the first flip-flop.
- a.k.a. a re-circulating shift register.

Johnson counter

does the same as ring counter but it uses the complement output of the last output (D-Flip-Flop).